Azimi Innovation Lab
AI Architect · End-to-End Projects

جلسه 12

کد تمیز و بهترین شیوه‌ها

چطور کدی بنویسیم و بخواهیم که خوانا و قابل نگهداری باشد

ادامه مستقیم جلسات قبل — Azimi Innovation Lab

Azimi Innovation Lab
AI Architect · End-to-End Projects

اتصال به جلسات قبل: شما حالا می‌توانید کد را بخوانید، تست کنید، با API و دیتابیس کار کنید و امنیت را رعایت کنید. قدم بعدی این است که کد تمیز و خوانا داشته باشید تا خودتان و دیگران (و حتی AI در آینده) راحت‌تر آن را بفهمند و تغییر دهند.

۱. کد تمیز چیست؟

کد تمیز کدی است که خواندنش آسان باشد، هدفش واضح باشد و تغییر دادنش سخت نباشد. حتی اگر AI کد را نوشته باشد، شما می‌توانید از آن بخواهید کد را تمیزتر کند.

قانون ساده: کدی بنویسید (یا بخواهید) که شش ماه بعد خودتان بتوانید بدون سردرگمی آن را بفهمید.

۲. نام‌گذاری خوب

اسم متغیرها، توابع و فایل‌ها باید واضح و توصیفی باشد.

```
// بد
let d = new Date();
function calc(a, b) { return a * b; }

// خوب
let currentDate = new Date();
function calculateTotalPrice(price, quantity) {
  return price * quantity;
}
```

اسم خوب خودش توضیح می‌دهد و نیاز به کامنت اضافه را کم می‌کند.

۳. توابع کوچک و متمرکز

هر تابع باید یک کار مشخص انجام دهد. تابع خیلی طولانی (بیش از ۲۰–۳۰ خط) معمولاً نشانه این است که باید شکسته شود.

```
// بهتر است به چند تابع کوچک تقسیم شود
function handleFormSubmit() {
  if (!validateForm()) return;
  const data = getFormData();
  saveData(data);
  showSuccessMessage();
}
```

۴. کامنت‌های مفید

کامنت باید «چرا» را توضیح دهد، نه «چه». اگر کد واضح باشد، کامنت زیاد لازم نیست.

```
// بد: مقدار را افزایش می‌دهد
count = count + 1;

// خوب: چون کاربر یک آیتم جدید اضافه کرده
count = count + 1;
```

۵. ثبات و سازماندهی

- از یک سبک نام‌گذاری استفاده کنید (مثلاً camelCase).
- فایل‌ها را منطقی نام‌گذاری و پوشه‌بندی کنید.
- کد تکراری را به تابع تبدیل کنید.
- فاصله‌گذاری و تورفتگی را یکدست نگه دارید.

۶. چطور از AI کد تمیزتر بخواهیم؟

به جای گفتن «کد بنویس»، این‌طور درخواست کنید:

- «این کد را تمیزتر و خواناتر کن.»
- «توابع را کوچک‌تر کن و اسم‌های بهتری بگذار.»
- «کامنت‌های مفید اضافه کن که چرا این کار انجام شده.»
- «کد تکراری را حذف کن و به تابع تبدیل کن.»

بعد از دریافت کد تمیز، حتماً آن را اجرا و تست کنید تا مطمئن شوید رفتارش تغییر نکرده است.

۷. نشانه‌های کد غیرتمیز

- متغیرهایی با اسم‌های یک‌حرفی (a, x, temp)
- توابع خیلی طولانی
- کد تکراری در چند جا

- نبود ساختار واضح در فایل‌ها
- کامنت‌های قدیمی که دیگر درست نیستند

۸. تمرین‌های عملی

تمرین ۱: بهبود نام‌گذاری

از AI یک کد ساده بگیرید. سپس از آن بخواهید اسم متغیرها و توابع را واضح‌تر کند. دو نسخه را مقایسه کنید.

تمرین ۲: شکستن تابع طولانی

از AI بخواهید یک تابع نسبتاً طولانی بنویسد، سپس همان را به ۳–۴ تابع کوچک‌تر تقسیم کند.

تمرین ۳: درخواست کد تمیز از اول

یک ویژگی کوچک (مثلاً اعتبارسنجی فرم) را از AI بخواهید و در پرامپت تأکید کنید که کد تمیز، با اسم‌های خوب و توابع کوچک باشد.

۹. چک‌لیست پایان جلسه

- می‌توانم تفاوت کد تمیز و غیرتمیز را توضیح دهم.
- اهمیت نام‌گذاری خوب را فهمیده‌ام.
- می‌دانم چطور از AI بخواهم کد را تمیزتر کند.
- حداقل یک کد را از نظر خوانایی بررسی کرده‌ام.
- تمرین‌ها را انجام داده‌ام.

۱۰. خلاصه و پل به جلسات بعد

در این جلسه یاد گرفتید که کد تمیز خواندن، نگهداری و توسعه پروژه را آسان‌تر می‌کند. حالا می‌توانید از AI کد بهتری بخواهید و خودتان هم کیفیت کد را ارزیابی کنید.

جلسات بعدی بیشتر روی پروژه‌های واقعی، پورتفولیو و موضوعات پیشرفته‌تر تمرکز خواهند داشت.

Azimi Innovation Lab
AI Architect · End-to-End Projects

پایان جلسه 12 — Azimi Innovation Lab

حالا می‌توانید کدهای خواناتر و حرفه‌ای‌تری داشته باشید. موفق باشید!